

iOS Development With Swift 4

Academy of Computing Machinery

Illya Starikov

What We'll Be Covering

1. The Model View Controller (MVC) Paradigm
2. Swift 4
3. Xcode
4. Interface Builder

Massive View Controller I

Err, Model View Controllers*

Model An object representing the data or user activity.

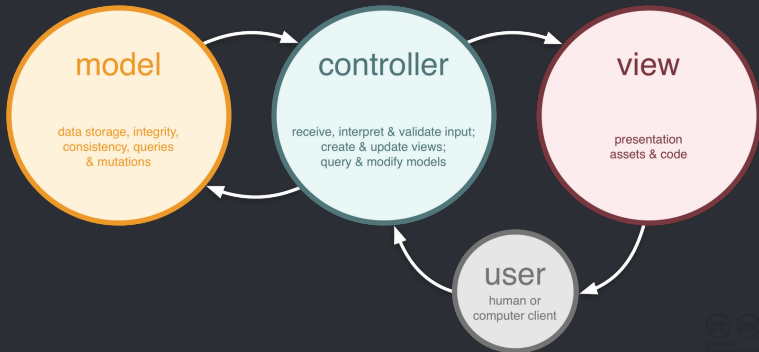
View A visual representation of the model, for user interaction.

Controller The interface between the model and the view.

Note the model should never talk to the view, and vice versa. Everything goes through the controller.

View Controller II

A more visual explanation



Swift 4

A brief introduction

```
print("Hello, Swift")
```

Swift 4

A brief Introduction

- Swift is a modern programming language with modern programming paradigms.
 - Closures
 - Tuples/multiple return values
 - First class functional programming support (i.e. , map, reduce, filter)
- Swift is open source, with all contents on available on <https://github.com/apple/swift>.
- Source compatible from Swift 3.
- Designed for safety (i.e. , automatic memory management (via reference counting), overflow checking, everything is utilized before use).

Variables

Swift Syntax I

- Variables in swift are declared with the keyword **var**.
- Constants are declared with the keyword **let**.
 - All variables that do not change *should be marked as constant*.
 - There are compiler warning for not using constants.
- Type inference is not only supported, but highly recommended.
- Swift is a *strongly typed* language.
 - This means when you declare a type, it stays that type.
 - Arithmetic operations should be performed on the same type. (i.e., `Int × Int` is okay, but `Int × Double` is not.)
 - Although Swift does support the idea of casting, it is often better to use constructors to get a new variable of desired type.

Variables (Example)

Swift Syntax I

```
let pi = 3.14 // Double
```

```
var answer = 420 // Int
```

```
answer /= 10 // oops
```

```
var piIsAnswerInt: Int // explicit type
```

```
piIsAnswerInt = Int(pi) * answer
```

```
print(piIsAnswerInt) // prints 126
```


Optionals

Swift Syntax II

Optionals are another type in Swift. There are two cases for optionals, either they are `nil` or a value. The syntax for an optional is `type?`, with the question mark signifying the optional. To unwrap an optional, i.e., get its value, the syntax is `variable!`.

*Unwrapping optionals **will cause a runtime error.***

Optionals (Example)

Swift Syntax II

```
var ACMPresident: String? = "Dalton Cole"
```

```
// this is unsafe, causes several warnings
```

```
print(ACMPresident) // prints Optional("Dalton Cole")
```

```
ACMPresident = nil
```

```
print(ACMPresident) // prints nil
```

```
print(ACMPresident!) // causes runtime error
```

```
ACMPresident = "Eric Michalak"
```

```
print(ACMPresident!) // prints Eric Michalak
```

Optionals

Swift Syntax III

Optionals are so prevalent in Swift, there is even special syntax for unwrapping them, `if let`. Most optionals will get unwrapped this way.

```
var ACMPresident: String? = nil
```

```
if let president = ACMPresident {  
    print("This current president is \(president).")  
} else {  
    print("ACM Elections are probably underway.")  
}
```

Data Structures

Swift Syntax IV

Swift supports all basic data structures you might need:

- Strings (`String`)
- Arrays (`Array<Character>`)
- Sets (`Set<Int>`)
- Dictionaries (`Dictionary<Int>`)
- Tuples (`Tuple<Int>`)

Enumerated Types (Enums)

Swift Syntax V

Enumerated types are just like like enums in other languages.

```
enum Direction {  
    case north  
    case south  
    case east  
    case west  
}
```

Enumerated Types (Example)

Swift Syntax VI

But because of Swift's type inference, enums have some brevity to them.

```
let northPoleDirection = Direction.north

if northPoleDirection == .north {
    print("Hello, Cold.")
} else if northPoleDirection == .south {
    print("When did penguins get here?")
} else {
    print("This definetly can't be right.")
}
```

Control Flow

Swift Syntax VII

Swift supports all paradigm of control flow, along with more modern loop methods.

- If-Else with `if...else`
- Switch-Cases with `switch { case ...: }`. Note that *switches must be exhaustive*, as in all switch cases must either go through all possible values or have a default.
- For-In loops via `for ... in ...`.
- Enumerated for loops via `for index in start...end`, where start is the start index and end is the end index (inclusive).
- While loops with `while`.
- Do-While loops with `repeat...while`

Control Flow (Example)

Swift Syntax VII

```
let answer = 42
```

```
if answer == 42 {  
    // prints "The answer to the question of life,  
    the universe and everything is 42"  
    print("The answer to the question of life, the  
    universe and everything is \(answer)")  
}
```

```
for i in 1...answer {  
    print(i) // prints 1 to 42  
}
```


Control Flow (Example)

Swift Syntax VII

```
let bestPokemon = ["pikachu", "wobbuffet", "mewtwo"]  
  
for pokemon in bestPokemon {  
    print(pokemon) // prints pikachu, wobbuffet,  
                    mewtwo  
}
```

Control Flow (Example)

Swift Syntax VII

```
for pokemon in bestPokemon {  
    switch pokemon {  
        case "pikachu":  
            print("Pika Pika")  
  
        case "wobbuffet":  
            print("WOBBUFFET!")  
  
        default:  
            print("Pokedex 404")  
    }  
}
```

Functions

Swift Syntax VIII

Functions have the following syntax

```
func function(external internal: Type) ->  
    return values {  
    ...  
    return 42  
}
```

Where **external** and **internal** are the parameter names. Internal parameter names are used inside the function, while external parameter names are used when calling the function. This will make more sense in a second.

Functions (Example)

Swift Syntax VIII

```
func calculateRadian(fromAngle angle: Int) -> Double
{
    let pi = 3.14
    return Double(angle)*(pi/180.0)
}
```

```
let radian = calculateRadian(fromAngle: 180)
```

Closures

Swift Syntax IX

Closures are just functions that act like variables. Because they act like variables, they can get additional data from the environment (this will become more clear in the example). Just as variables have types, so do closures: (parameter types) -> return type. The general syntax of a Swift closures is

```
{ (parameters) -> return type in  
    ...  
}
```

Closures (Examples)

Swift Syntax IX

Take the following function.

```
func createCounter(startingAt start: Int,  
    incrementing incrementor: Int) -> () -> Int {  
    var value = start  
  
    return { () -> Int in  
        value += incrementor  
        return value  
    }  
}
```

Closures (Examples)

Swift Syntax IX

It's a function that takes in a start value and how much to increment by, and returns a closure (or, another function) that increments by the value passed in. So it would be used like so.

```
let counter = createCounter(startingAt: 42,  
    incrementing: 3)  
  
for _ in 1...5 {  
    print(counter()) // prints 45, 48, 51, 54, 57  
}
```

Note the underscore just represents that the program doesn't require that value.

Classes and Structs

Swift Syntax X

Swift is a purely object-oriented language, naturally classes and structs are present. Classes and structs adhere to the four pillars of object oriented programming:

1. Abstraction
2. Encapsulation
3. Inheritance
4. Polymorphism

The only difference between structs and classes is:

struct Value type. All instances are copied when being passed around.

class Reference type, i.e., pass by reference. A pointer to all instances.

Classes and Structs

Swift Syntax X

Some notes on classes and structs.

- Classes require all values to be initialized upon instantiation, whether it be through the constructor or default property values. Structs are not required to do this.
- Both classes and structs can have properties and methods.
- Structs and classes have very different rules on initializers.

Classes and Structs (Example)

Swift Syntax X

```
class Zombie {  
    var health = 100  
    var attackBoost = 2 // valued from 1-5  
    var defenseBoost = 1 // valued from 1-5  
  
    func getAttackValue() -> Int {  
        return 4 // chosen by fair dice roll  
    }  
  
    func attack(_ zombie: Zombie) {  
        zombie.health -= getAttackValue() * attackBoost  
    }  
}
```

Classes and Structs

Swift Syntax X

Everything is familiar, even using this `Zombie` object is similar to most programming language.

```
let rick = Zombie(), morty = Zombie()  
rick.attack(morty)
```

However, this isn't good programming practice. Some things should be marked as `private`, and some `public`. In swift `public` and `private` are reserved words to mark things as public and private. In Swift, getters and setters get first class support with computed properties.

Classes and Structs (Public & Private)

Swift Syntax X

```
class Zombie {  
    var health = 100  
    private var attackBoost = 2 // valued from 1-5  
    private var defenseBoost = 1 // valued from 1-5  
  
    private func getAttackValue() -> Int {  
        return 4 // chosen by fair dice roll  
    }  
  
    public func attack(_ zombie: Zombie) {  
        zombie.health -= getAttackValue() * attackBoost  
    }  
}
```

Classes and Structs (Computed Properties)

Swift Syntax X

```
class Zombie {  
    private var myHealth = 100  
  
    public var health: Int {  
        get {  
            return myHealth  
        }  
    }  
    ...  
    public func attack(_ zombie: Zombie) {  
        let damage = getAttackValue() * attackBoost  
        zombie.take(damage: damage)  
    }  
  
    public func take(damage: Int) {  
        myHealth -= damage  
    }  
}
```

Classes and Structs (Constructors)

Swift Syntax X

As in most programming language, Swift has different rules about different constructors. Very rarely will writing a new constructor be necessary, but to write a convenience constructor. This has the syntax:

```
convenience init(parameters) {  
    self.init()  
    ...  
}
```

Classes and Structs (Example)

Swift Syntax X

```
class Zombie {  
    private var attackBoost = 2 // valued from 1-5  
    private var defenseBoost = 1 // valued from 1-5  
  
    convenience init(attackBoost: Int, defenseBoost: Int)  
    {  
        self.init()  
  
        self.attackBoost = attackBoost  
        self.defenseBoost = defenseBoost  
    }  
    ...  
}
```

Classes and Structs (Example)

Swift Syntax X

So now there is the default constructor, and the convenience initializer.

```
let rick = Zombie(attackBoost: 5, defenseBoost: 1)
```

```
let morty = Zombie()
```

```
rick.attack(morty)
```


The Rest of Swift

Swift Syntax XI

This was just a brief introduction to Swift — it still has many more powerful features:

- Type Casting
- Nesting Type
- Extensions
- Protocols
- Generics

To learn more about these features, and to go more in depth about the previous talking points, check out The Swift Programming Language on [Apple's Developer Site](#).

In Closing

Spam Illya with all questions, comments, and insults.

 @IllyaStarikov

 IllyaStarikov

 starikov@mst.com

Credits

- Model View Controller graphic courtesy of Basics of Web Development at <https://basicsofwebdevelopment.wordpress.com>